

# BOT GPT

AI Engineer Case Study · BOT Consulting

NestJS

PostgreSQL + pgvector

Gemini 2.5 Flash

LangChain

Next.js 15

Aman Pawar · 2026

# What We Built

Full-stack AI chat platform — two conversation modes:

### Open Chat

Streaming conversations with Gemini 2.5 Flash. Sliding window or summarization context strategy per conversation.

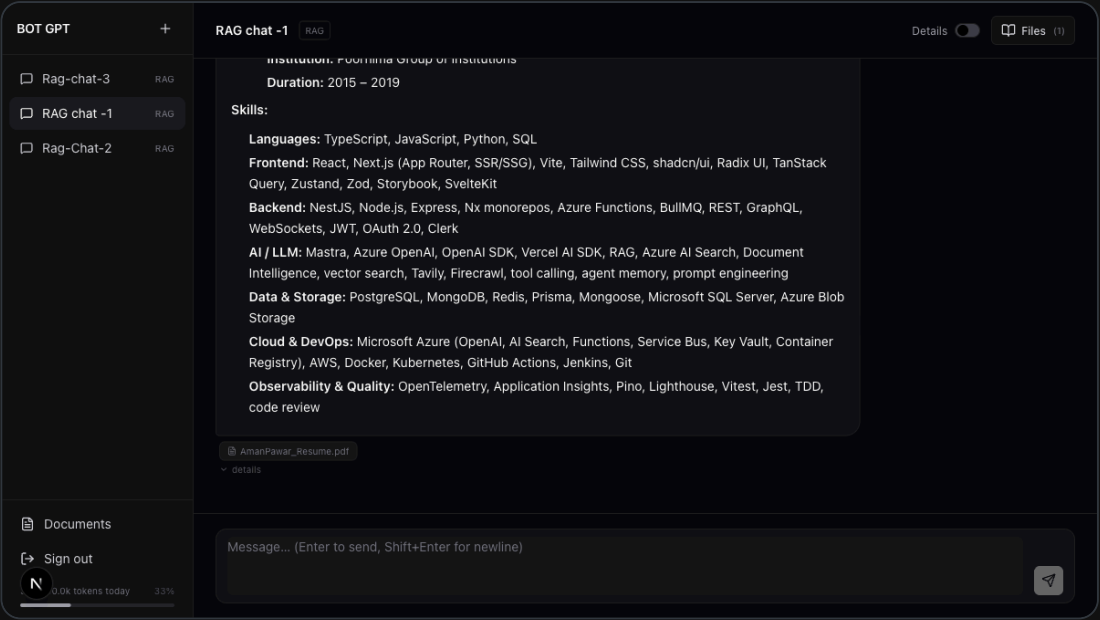
### RAG Chat

Attach PDFs from your library → embedded into pgvector → every answer grounded in your documents, not training data.

### Production features

JWT auth · daily token limits (100k/day) · exponential backoff retry · in-memory caching · markdown rendering · toast notifications · multi-stage Docker · GitHub Actions CI

Live app



# Architecture

```
Browser
  |   HTTPS · REST + SSE
NestJS API
  |─ JwtAuthGuard (every route)
  |─ ConversationsController
  |   |─ AiService ← LangChain + Gemini
  |   |─ ConversationsService
  |       |─ context window |─ token limit |─ cache
  |─ DocumentsController
  |   |─ AiService ← embed + pgvector
  |─ AuthController → bcrypt · JWT
  |
  ▼
PostgreSQL users · conversations · messages
           documents · conversation_documents
           doc_embeddings (vector)

Cloudflare R2 raw PDFs

Gemini API  gemini-2.5-flash + gemini-embedding-2
```

- ✓ **Single DB** — pgvector inside Postgres. No Pinecone/Weaviate.
- ✓ **Stateless API** — JWT, no session store. Scales horizontally.
- ✓ **SSE not WebSocket** — unidirectional, HTTP/1.1, perfect for token streaming.
- ✓ **Document library** — documents live in user library; conversations attach/detach them. Many-to-many via `conversation_documents`.

# Auth + Data Model

```
POST /auth/register { email, password }  
  └─ bcrypt.hash(password, 10)  
  └─ INSERT INTO users (409 if email taken)  
  └─ jwt.sign({ sub: userId }) → { access_token }
```

```
POST /auth/login { email, password }  
  └─ bcrypt.compare → 401 if wrong  
  └─ jwt.sign(...) → { access_token }
```

Every route:

```
Authorization: Bearer <token>  
  └─ JwtAuthGuard → req.user = { id, email }  
      401 if missing / expired / tampered
```

## Schema (key tables):

|                        |                                                                        |
|------------------------|------------------------------------------------------------------------|
| users                  | (id, email, password_hash)                                             |
| conversations          | (id, user_id, title, mode,<br>context_strategy, token_count)           |
| messages               | (id, conversation_id, role,<br>content, token_count,<br>sources JSONB) |
| documents              | (id, user_id, filename,<br>r2_url, status)                             |
| conversation_documents | (conversation_id, document_id,<br>attached_at) ← join table            |
| doc_embeddings         | (id, content, vector(768),<br>metadata JSONB)                          |

Documents are *not* tied to a conversation — they belong to the user's library and get attached/detached as needed.



**BOT GPT**

Conversational AI with open chat and document-grounded RAG mode.

- Open chat on any topic
- Upload PDFs and ask questions about them
- Context-aware memory with sliding window or summarization



**Sign in**

Enter your credentials to continue

Email

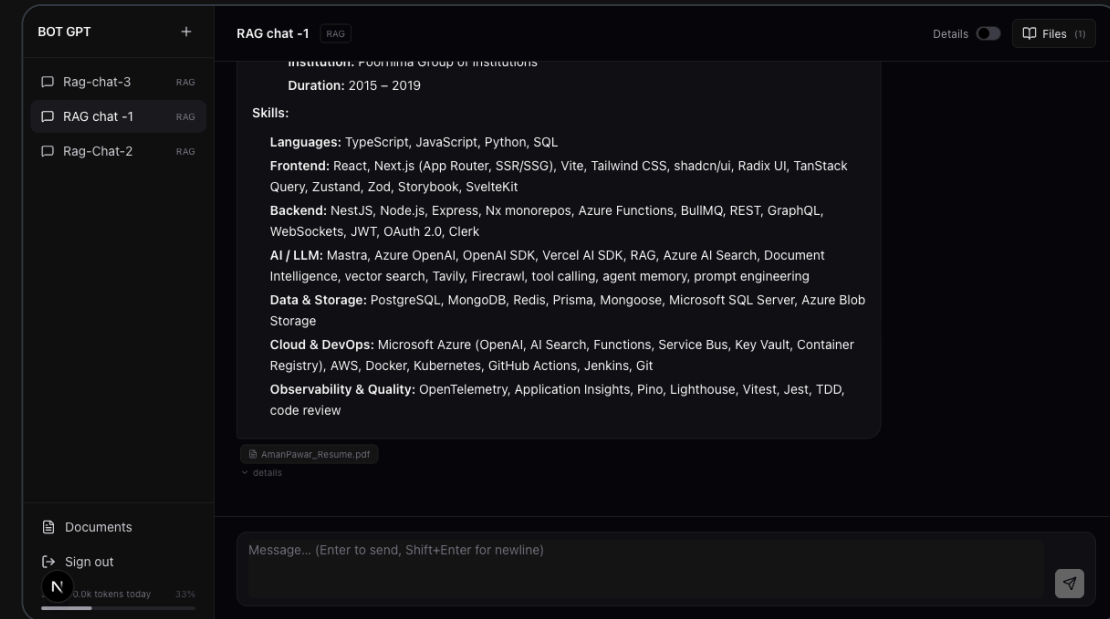
Password

No account? [Create one](#)

# LLM Streaming via SSE

```
POST /conversations/:id/messages { content }
|
|— getDailyTokensUsed(userId)
|   429 JSON if ≥ 100 000 tokens/day
|   (before SSE headers — still JSON-switchable)
|
|— buildContextWindow(messages, budget)
|   budget = 8 000 (open) | 6 000 (RAG)
|
|— res.setHeader('Content-Type', 'text/event-stream')
|
|— for await (token of llm.stream(...))
|   res.write(`data: {"token":"..."}\n\n`)
|
|— res.write(`data: {"done":true,
|   "userTokens":12, "assistantTokens":87,
|   "sources":[{"documentId, filename}]}"\n\n`)
|   res.end()
```

RAG is an AI technique that augments



## Why 429 before SSE?

Once `text/event-stream` is written, the response is locked — can't switch to JSON error. Daily limit checked first.

## Sources on done event

RAG source citations returned in the `done` event and persisted as JSONB on the message. Frontend renders filename badges.

# Context Strategies

## Sliding Window (default)

```
budget = 8 000 t (open) | 6 000 t (RAG)
```

```
walk newest → oldest:
```

```
msg5  80 t   total=80   ✓  
msg4  120 t  total=200  ✓  
msg3  7400 t total=7600 ✓  
msg2  900 t  total=8500 ✗ stop
```

```
window = [msg3, msg4, msg5]  chronological
```

**Edge:** msgs alone > budget?

→ include it anyway (min-1 guarantee)

### Zero extra API calls

Guaranteed most-recent context. Old turns silently dropped when history overruns budget.

## Summarization

Step 1 first-pass window

```
older = messages before window
```

Step 2 summarize older slice (1 LLM call)

```
summary = llm.invoke(older)
```

```
summaryTokens = estimateTokens(summary)
```

Step 3 rebuild window

```
recent = buildWindow(budget - summaryTokens)
```

```
← total never overruns budget
```

Step 4 send to Gemini:

```
[ SystemMessage,  
  AssistantMessage("[Summary]: ..."),  
  ...recent..., HumanMessage ]
```

### Trade-off

1 extra LLM call per request when history is long, but preserves semantic content of old turns (80-90% token reduction).

# RAG Pipeline

## Ingest path

POST /documents { PDF }

↓ Upload → Cloudflare R2 (S3-compatible, free egress)

↓ RecursiveCharacterTextSplitter  
chunkSize:1000 · overlap:200

↓ GoogleGenerativeAIEmbeddings → VECTOR(768)

↓ PGVectorStore · metadata: { document\_id, filename }

POST /conversations/:id/documents/:docId

↓ INSERT conversation\_documents

## Query path

POST /conversations/:id/messages

↓ getAttachedDocuments(convId) → [docId...]

↓ Parallel similaritySearch per document\_id  
perDoc = ceil(topK / docs.length)

↓ "Context:\nchunk1\n---\nn...\nAnswer: {q}"

↓ withRetry(() → llm.stream(messages))

↓ done event → sources persisted as JSONB

Filter by document\_id (not conversation) — documents can be shared across conversations with no vector duplication.



# Document Library

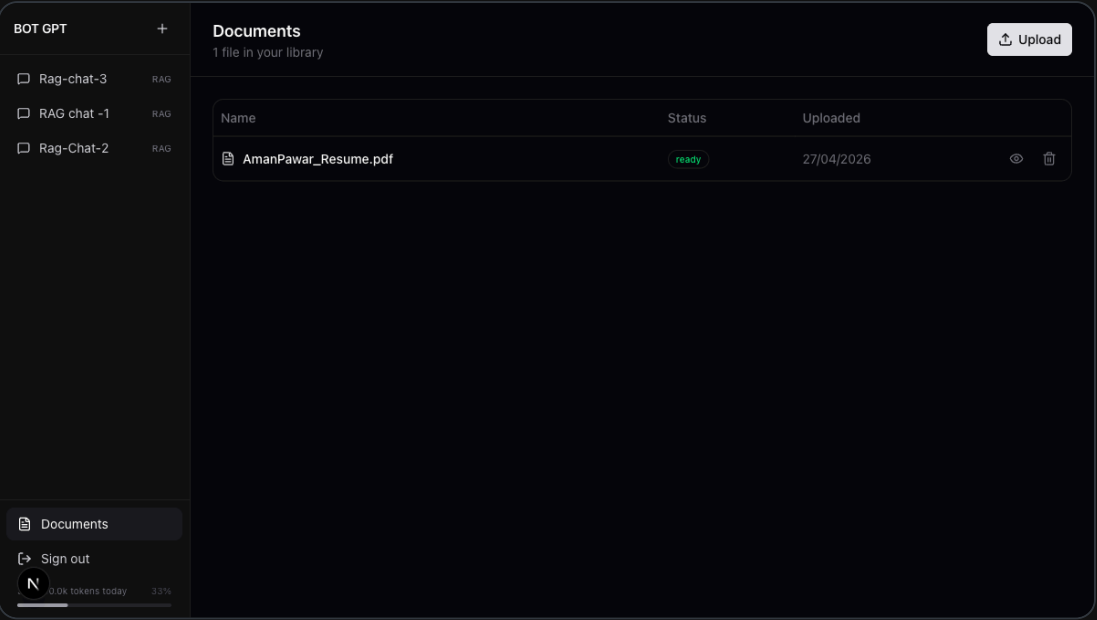
Documents belong to the **user library**, not a conversation. Conversations attach/detach documents as needed.

```
POST    /documents          upload to library
GET     /documents          list user library
DELETE  /documents/:id      delete + R2 object + vectors

GET     /conversations/:id/documents  list attached
POST    /conversations/:id/documents/:docId  attach
                                           409 if already attached
DELETE  /conversations/:id/documents/:docId  detach
```

## Why library model?

A document uploaded once can be attached to multiple conversations without re-uploading or re-embedding. Detach removes the join row only — embeddings stay.



## Cloudflare R2

S3-compatible, zero egress fees. DeleteObjectCommand called on document delete — no orphaned objects.

# Reliability & Cost Controls

## Token Limits

```
getDailyTokensUsed:
  SUM(msg.token_count)
  JOIN conversations
  WHERE user_id = ?
  AND created_at ≥ UTC day

if used ≥ 100 000:
  res.status(429).json(...)
  ← before SSE headers

After stream:
  saveMessage(..., userTokens)
  saveMessage(..., assistantTokens)
  token_count += total
```

### UTC midnight reset

startOfDay.setUTCHours(0,0,0,0) — consistent across timezones. Configurable via `DAILY_TOKEN_LIMIT` env.

## LLM Retry

```
withRetry(fn, maxAttempts=3):
  delay = 1000ms

  attempt 1 → fail → sleep 1s
  attempt 2 → fail → sleep 2s
  attempt 3 → OK → stream

  all 3 fail → throw
    → SSE: {"error":"LLM unavailable"}
    → toast notification on client

Applied to:
  streamOpenChat()
  streamRagChat()
```

### Tested with Jest

Mock `setTimeout` to instant. Fail twice then succeed. Assert 3 calls, correct tokens returned.

## Caching

```
GET /conversations?page&limit

key = `convlist:${userId}:${page}:${limit}`

cache.get(key) → hit: return
                → miss: query DB
                        cache.set(key, ttl=60s)

Invalidation:
  create() → cache.del(`convlist:${userId}`)
  remove() → cache.del(`convlist:${userId}`)
```

### In-memory now / Redis later

Zero infra for single instance. Swap to Redis store when scaling — zero service code changes.

# Testing: TDD Throughout

RED → GREEN → commit. 40 tests · 5 suites · <2 s. Zero network or DB needed.

| Suite                      | Key scenarios                                                                                                                                                   | #  |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| auth.service.spec          | Register (duplicate→409), login (bad pw→401), JWT issued                                                                                                        | 5  |
| conversations.service.spec | Create, pagination, ownership 403/404, sliding window budget (incl. oversized edge case), daily token SUM query                                                 | 14 |
| documents.service.spec     | Upload + embed, list by user, delete + pgvector chunk cleanup                                                                                                   | 5  |
| ai.service.spec            | estimateTokens, streamOpenChat, streamRagChat context injection, retry (2 fails→success; all 3 fail), summarizeHistory, retrieveRelevantDocs (parallel per-doc) | 16 |

All external deps (Gemini, pgvector, TypeORM repos, S3) are Jest-mocked. Retry tests mock `setTimeout` — no real waits.

## Multi-Stage Dockerfile

```
# Stage 1: deps - layer-cached
FROM node:20-alpine AS deps
COPY package*.json ./ && RUN npm ci

# Stage 2: builder
FROM node:20-alpine AS builder
COPY --from=deps /app/node_modules .
COPY . . && RUN npm run build

# Stage 3: runner - minimal image
FROM node:20-alpine AS runner
ENV NODE_ENV=production
RUN adduser --system --uid 1001 nestjs
COPY --from=builder /app/dist ./dist
COPY --from=deps /app/node_modules .
USER nestjs # non-root
EXPOSE 3001
CMD ["node", "dist/main"]
```

### Non-root user

uid 1001 cannot write to system dirs if the container is compromised.

## GitHub Actions CI

```
on: [push, pull_request]

jobs:
  lint:
    steps:
      - run: npm ci
      - run: npm run lint # ESLint + TS check

  test:
    steps:
      - run: npm ci
      - run: npm test # Jest - 40 tests
```

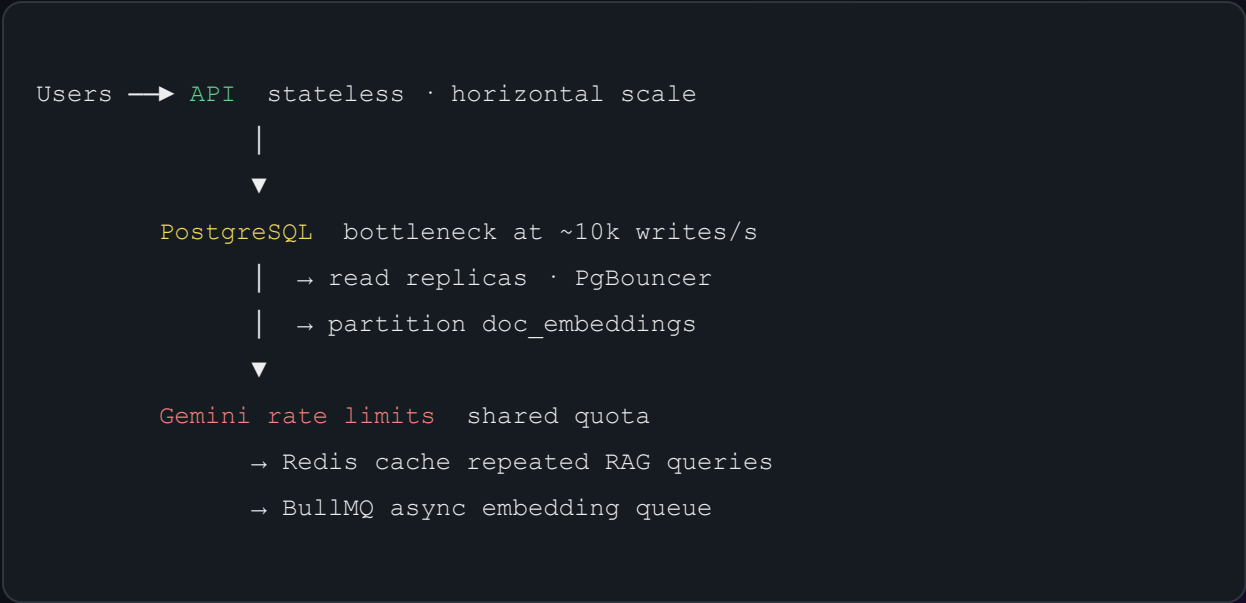
### Lint + test in parallel

Two independent jobs run concurrently on every push and PR. Blocks merge on any failure.

### docker-compose

`docker-compose up --build` starts postgres (pgvector) + backend together.

# Scalability & Summary



**Cache → Redis**

Swap `CacheModule.register()` config. Zero service changes. Shared cache across instances.

- ✓ JWT auth — register + login
- ✓ SSE streaming · typing animation
- ✓ RAG — pgvector + Gemini embeddings
- ✓ Document library + attach / detach
- ✓ Sliding window + summarization strategies
- ✓ Daily token limit (100k/day · 429)
- ✓ Exponential backoff retry (3×)
- ✓ In-memory cache · invalidation
- ✓ Markdown rendering · toast notifications
- ✓ Multi-stage Docker (non-root)
- ✓ GitHub Actions CI · 40 tests TDD

NestJS

PostgreSQL + pgvector

Gemini 2.5 Flash

LangChain

Next.js 15

Docker

GitHub Actions